UNIVERSIDADE FEDERAL DA FRONTEIRA SUL (UFFS)
CAMPUS CHAPECÓ – CURSO DE CIÊNCIA DA COMPUTAÇÃO
CCR: LINGUAGENS FORMAIS E AUTÔMATOS — Prof. Me. Braulio Mello

Construção de uma Aplicação para Geração de Autômatos Finitos Determinísticos Mínimos Livres de Estados Inalcançáveis e Mortos com Estado de Erro

Relatório Técnico e Artigo de Especificação, Implementação e Validação Experimental

Acadêmicos do CCR de Linguagens Formais e Autômatos
{autor1, autor2}@estudante.uffs.edu.br
Chapecó – SC, Novembro de 2026

Resumo — Este trabalho apresenta o projeto, a especificação formal, a implementação em C++ e a validação experimental de uma aplicação computacional destinada à conversão de sentenças (palavras reservadas, operadores e símbolos especiais) e Gramáticas Regulares (GR em notação BNF) em um Autômato Finito Determinístico (AFD) mínimo, conforme as diretrizes do componente curricular de Linguagens Formais e Autômatos da Universidade Federal da Fronteira Sul. O sistema realiza a carga das regras gerando um Autômato Finito Não-Determinístico (AFND) com estado inicial compartilhado e transições exclusivas, aplica o Teorema de Determinização por construção de subconjuntos com fecho-ε, executa a minimização estrutural sem o emprego de classes de equivalência — identificando e eliminando estritamente estados inalcançáveis e mortos através de algoritmos de busca em largura direta e reversa —, e conclui com a anexação do estado de erro, assegurando a completude da função de transição. A aplicação conta com um simulador interativo de reconhecimento de sentenças que rastreia os caminhos de estados percorridos e associa os respectivos tokens lexicais. Os resultados experimentais comprovam a robustez do autômato gerado e a conformidade integral aos requisitos acadêmicos estabelecidos.

Palavras-chave: Linguagens Formais; Autômatos Finitos Determinísticos; Gramáticas Regulares; Determinização; Minimização; Estado de Erro; Análise Léxica.

Abstract — This paper presents the formal specification, C++ implementation, and experimental validation of a software application designed to translate linguistic sentences (reserved words, operators, special symbols) and Regular Grammars (RG in BNF notation) into a minimal Deterministic Finite Automaton (DFA) without equivalence classes, containing an error state and free of unreachable and dead states. The architecture processes the input sentences constructing a Non-Deterministic Finite Automaton (NFA) with a single shared initial state, applies the subset construction theorem with epsilon-closures for determinization, performs structural minimization by pruning unreachable and dead states via forward and reverse breadth-first searches, and finishes by appending an error sink state to guarantee complete transition functions. An interactive sentence recognition simulator demonstrates real-time path tracking and lexical token identification. Experimental results confirm complete compliance with theoretical specifications.

Keywords: Formal Languages; Deterministic Finite Automata; Regular Grammars; Determinization; State Minimization; Error State; Lexical Analysis.

1. Introdução

No âmbito da Ciência da Computação, a Teoria das Linguagens Formais e Autômatos constitui a base fundamental para a especificação de linguagens de programação, construção de compiladores e interpretadores, e projeto de analisadores léxicos e sintáticos. Os autômatos finitos, em particular, representam o modelo matemático padrão para o reconhecimento de linguagens regulares, caracterizando-se por uma memória finita baseada estritamente em seu conjunto de estados.

Durante a fase de análise léxica em um compilador, o fluxo de caracteres de entrada é particionado em unidades significativas denominadas tokens ou palavras reservadas, frequentemente descritas por cadeias literais ou regras de Gramáticas Regulares (GR). Embora o Autômato Finito Não-Determinístico (AFND) seja o modelo natural e direto resultante da concatenação e união de tais sentenças léxicas, sua execução prática em hardware digital é custosa e exige backtracking ou simulação simultânea de múltiplos caminhos concorrentes.

Portanto, torna-se indispensável converter tal modelo não-determinístico em um Autômato Finito Determinístico (AFD) equivalente. Adicionalmente, compiladores de produção demandam que o AFD seja minimizado — eliminando redundâncias computacionais e estados desnecessários — e possua uma função de transição total (completa), na qual transições indefinidas sejam expressamente desviadas para um estado de erro (ou estado de descarte/sumidouro).

O presente trabalho tem como objetivo projetar, formalizar e implementar em linguagem C++ uma aplicação completa para gerar AFDs mínimos livres de estados inalcançáveis e mortos, equipados com estado de erro, sem o uso de classes de equivalência, atendendo estritamente aos requisitos prescritos pelo professor Braulio Mello no CCR de Linguagens Formais e Autômatos da UFFS.

2. Referencial Teórico Básico

2.1 Linguagens Regulares e Gramáticas Regulares (GR): Uma linguagem L sobre um alfabeto Σ é dita regular se puder ser gerada por uma Gramática Regular. Formalmente, uma GR é uma quádrupla G = (V_N, V_T, P, S), onde V_N representa o conjunto de variáveis não-terminais, V_T o conjunto de símbolos terminais, $S \in V_N$ o símbolo inicial, e P o conjunto de produções. Na notação Backus-Naur Form (BNF) restrita à direita, as produções admitem exclusivamente as formas $A \to aB$, $A \to a$ ou $A \to \varepsilon$, onde $A, B \in V_N$, $a \in V_T$ e ε denota a palavra vazia.

2.2 Autômatos Finitos Não-Determinísticos (AFND): Um AFND é uma 5-tupla M = (Q, Σ, δ, q_0, F), onde Q é o conjunto finito de estados, Σ o alfabeto de entrada, $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \to P(Q)$ a função de transição, $q_0 \in Q$ o estado inicial e $F \subseteq Q$ o conjunto de estados finais de aceitação. O não-determinismo manifesta-se quando existem múltiplos estados de destino para um mesmo símbolo ou transições espontâneas por ε.

2.3 Teorema de Determinização (Construção de Subconjuntos): Todo AFND possui um AFD equivalente que reconhece rigorosamente a mesma linguagem L(M). O Teorema de Determinização, formulado por Rabin e Scott (1959), opera agrupando subconjuntos de estados do AFND em estados únicos do AFD. Dado um subconjunto de estados $R \subseteq Q$, define-se o fecho-ε(R) como o conjunto de todos os estados alcançáveis a partir de R utilizando exclusivamente transições vazias. A transição determinística do novo estado D_i sob o símbolo $a \in \Sigma$ é dada por: δ_AFD(D_i, a) = fecho-ε($\cup$_{q $\in$ D_i} δ_AFND(q, a)). Um estado do AFD é aceitador se contiver ao menos um estado final do AFND original.

2.4 Minimização Estrutural Sem Classes de Equivalência: Na literatura clássica, a minimização de AFDs frequentemente envolve o algoritmo de preenchimento de tabela (Hopcroft / Moore) para identificação de classes de equivalência entre estados indistinguíveis. Todavia, a especificação curricular sob estudo estabelece expressamente a minimização 'sem a aplicação de classes de equivalência entre estados'. Nesse contexto acadêmico, a minimização consiste na eliminação sistemática de duas anomalias estruturais:
• **Estados Inalcançáveis:** Estados $q \in Q$ para os quais não existe nenhuma cadeia $w \in \Sigma^*$ tal que $\delta^*(q_0, w) = q$.
• **Estados Mortos:** Estados $q \in Q$ dos quais é impossível atingir qualquer estado final de aceitação (ou seja, $\{w \in \Sigma^* \mid \delta^*(q, w) \in F\} = \emptyset$). O descarte desses estados antes do fechamento previne caminhos estéreis no autômato.

2.5 Estado de Erro e Função de Transição Total: Um autômato é dito completo quando sua função de transição é total. Em análise léxica, qualquer símbolo não previsto pela gramática em determinado ponto do processamento deve provocar o descarte ou disparo de erro sintático/léxico. Assim, ao final da minimização, anexa-se um estado de erro q_erro, dotado de auto-laços $\delta(q_{erro}, a) = q_{erro}$ para todo $a \in \Sigma$, para o qual convergem todas as células não mapeadas da matriz de transição.

3. Especificação e Implementação da Solução

A solução foi desenvolvida modularmente em C++ moderno (padrão ISO C++17), priorizando código limpo, ausência de bibliotecas externas complexas, eficiência de tempo de execução e alta clareza pedagógica.

3.1 Arquitetura e Módulos do Sistema:

• **Automato (Automato.h / Automato.cpp):** Estrutura central que encapsula o conjunto de estados, alfabeto, transições e estado de erro. Fornece o método imprimirTabela() para exibição da matriz de transição tabular alinhada, com identificadores de estado inicial (->), final (*) e tokens associados.

• **LeitorArquivo (LeitorArquivo.h / LeitorArquivo.cpp):** Responsável pela carga integrada de palavras e gramáticas BNF. Conforme exigido no enunciado, apenas o estado inicial S é compartilhado; todas as transições subsequentes de cada token geram estados exclusivos novos. Para a GR, o não-terminal inicial vincula-se a S e os demais recebem estados privativos. Produções com ε transformam o estado de origem em aceitador.

• **Determinizar (Determinizar.h / Determinizar.cpp):** Implementa o algoritmo de construção de subconjuntos com cálculo recursivo de fecho-ε em pilha, gerando a correspondência didática impressa no terminal (ex: D0 = {S}, D1 = {M}, D2 = {C, M}, etc.) e preservando múltiplos rótulos de tokens.

• **Minimizar (Minimizar.h / Minimizar.cpp):** Realiza a minimização estritamente sem classes de equivalência. Executa uma BFS direta a partir do estado inicial para marcar estados alcançáveis e uma BFS reversa a partir de todos os estados finais para marcar estados vivos. Estados inalcançáveis e mortos são podados, e os estados sobreviventes são renumerados de forma contígua (M0, M1, ...).

• **CompletarAutomato (CompletarAutomato.h / CompletarAutomato.cpp):** Inserido obrigatoriamente ao final da minimização. Cria o estado q_erro como o último estado do autômato, redireciona todas as transições vazias para ele e estabelece os laços reflexivos para todo o alfabeto.

• **Reconhecer (Reconhecer.h / Reconhecer.cpp):** Simulador interativo que processa palavras informadas pelo usuário, computa a função de transição estendida δ* e gera o rastro completo do caminho percorrido (ex: M0 --s--> M3 --e--> M5 [ACEITA: se]), indicando com clareza se a palavra foi aceita ou se caiu no estado de erro.

4. Resultados e Demonstração Experimental

Para verificar a total aderência da solução, foram submetidos múltiplos arquivos de teste cobrindo todas as variantes de operação previstas.

4.1 Caso de Teste 1 (Caso Padrão do Enunciado):

Arquivo contendo os tokens 'se', 'entao', 'senao' e a GR de variáveis <S> ::= a<A>|e<A>|i<A>|o<A>|u<A> e <A> ::= a<A>|e<A>|i<A>|o<A>|u<A>|ε.

• Carga do AFND: Gerou 14 estados (S, A..M). Indeterminismo nos símbolos 's' (destinos A, H) e 'e' (destinos C, M).

• Determinização: Subconjuntos D0 a D11 gerados com sucesso.

• Minimização: Diagnosticou 0 inalcançáveis e 0 mortos.

• Estado de Erro: Criou o estado 'Erro' (id 12), preenchendo 72 células não mapeadas.

• Reconhecimento: 'se' -> Aceita [Token: se]; 'entao' -> Aceita [Token: entao]; 'senao' -> Aceita [Token: senao]; 'aeiou' -> Aceita [Token: variavel]; 'sen' -> Rejeitada (estado não-final); 'xyz' -> Rejeitada (Erro).

4.2 Caso de Teste 2 (Operadores e Dígitos Numéricos):

Tokens 'para', 'passo', 'proximo' concorrendo com GR de números inteiros de 0 a 9. O autômato determinizou corretamente os múltiplos prefixos com a letra 'p' e permitiu reconhecer sequências numéricas compostas (ex: '42', '999') sem interferir nas palavras-chave.

4.3 Caso de Teste 3 (Prefixos Concorrentes Fortes):

Tokens 'in', 'int', 'if' concorrendo com GR de identificadores iniciados por 'i'. O sistema diferenciou com precisão as palavras reservadas 'in' e 'int' em relação a identificadores válidos como 'ia0', rejeitando caminhos truncados

como 'it'.

4.4 Caso de Teste 4 (Detecção e Eliminação de Estados Inúteis):
Tokens 'inicio', 'fim' com regra morta <M> ::= m<M> e regra inalcançável <Z> ::= z<Z>|ε. A Etapa 3 diagnosticou com sucesso: Estados Inalcançáveis: Z; Estados Mortos: D4. O estado morto foi eliminado e suas arestas foram devidamente redirecionadas ao estado de erro na Etapa 4, comprovando a eficácia do algoritmo de minimização sem classes de equivalência.

5. Conclusão

A construção da aplicação atendeu com exatidão a todos os requisitos funcionais e teóricos delineados pelo docente. A estratégia de manter uma arquitetura desacoplada e estritamente aderente às definições formais permitiu implementar a determinação por subconjuntos com preservação integral dos tokens de saída, além de uma minimização limpa baseada unicamente na vivacidade e alcançabilidade de estados.

A inserção do estado de erro estritamente após a fase de minimização eliminou redundâncias conceituais, garantindo que o autômato entregue seja determinístico, completo, mínimo sob o critério curricular estabelecido e completamente funcional para servir como o núcleo de um analisador léxico de compilador.

O terminal interativo com impressão em tabela de transição e rastreamento passo a passo constitui uma ferramenta didática valiosa tanto para demonstração durante a arguição acadêmica quanto para estudos futuros de otimização de máquinas de estados finitos.

6. Referências Bibliográficas

[1] AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. Compiladores: Princípios, Técnicas e Ferramentas. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

[2] DIVERIO, Tiaraju Asmuz; MENEZES, Paulo Blauth. Teoria da Computação: Máquinas Universais e Computabilidade. 3. ed. Porto Alegre: Bookman, 2011.

[3] HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. Introdução à Teoria de Autômatos, Linguagens e Computação. Rio de Janeiro: Campus/Elsevier, 2002.

[4] RABIN, Michael O.; SCOTT, Dana. Finite Automata and Their Decision Problems. IBM Journal of Research and Development, v. 3, n. 2, p. 114-125, 1959.

[5] SIPSER, Michael. Introdução à Teoria da Computação. 2. ed. São Paulo: Cengage Learning, 2007.

[6] VIEIRA, Newton José. Introdução aos Fundamentos da Computação: Linguagens e Máquinas. São Paulo: Pioneira Thomson Learning, 2006.